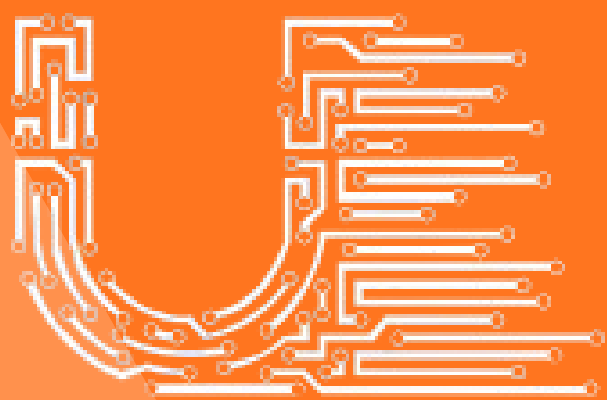




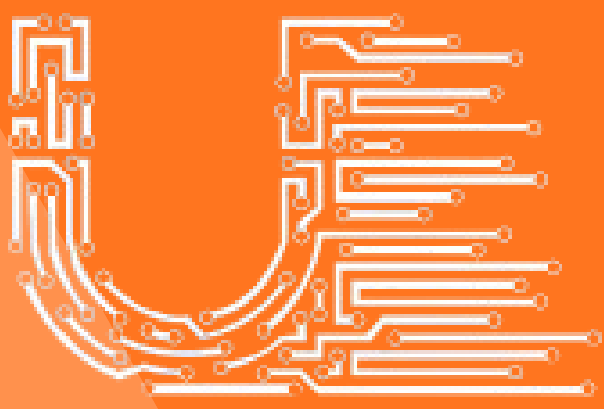
**UDYAM' 26**

# **I-CHIP '26**

## **PS-1**

**ELECTRONICS  
ENGINEERING  
SOCIETY**





UDYAM' 26

# SCALE-X

A Fixed-Point Image Scaling Challenge

**Image scaling** is a fundamental operation in digital image processing and is widely used across modern electronic systems, including display pipelines, video processing engines, camera systems, medical imaging equipment, automotive vision, and multimedia accelerators. In many such applications, images must be resized to match display resolutions, sensor formats, or algorithmic requirements while preserving visual quality and minimizing artifacts.

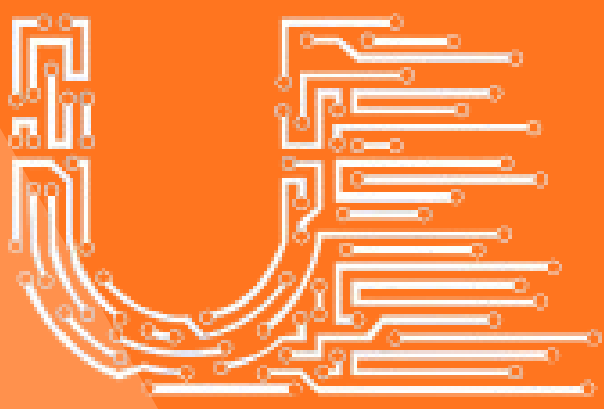
Although image scaling can be implemented easily in software, real-time and power-constrained systems demand dedicated hardware solutions. Software-based approaches often suffer from high latency, excessive memory bandwidth usage, and increased power consumption, making them unsuitable for embedded and high-performance applications. As a result, image scaling is frequently implemented as a specialized hardware accelerator within a larger System-on-Chip (SoC) or FPGA-based design.

Among various scaling techniques, **bilinear interpolation** provides a favorable balance between computational complexity and image quality. Compared to nearest-neighbor interpolation, bilinear interpolation significantly reduces blockiness and aliasing artifacts by considering the contribution of four neighboring pixels.

**ELECTRONICS  
ENGINEERING  
SOCIETY**







**UDYAM' 26**

At the same time, it remains considerably simpler than higher-order techniques such as bicubic interpolation, making it well suited for efficient hardware implementation.

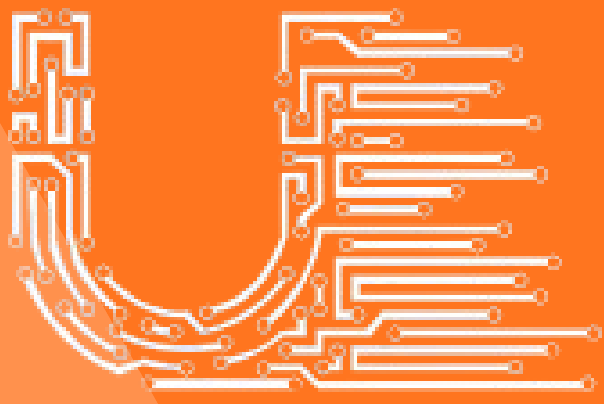
In hardware design, implementing bilinear interpolation presents several non-trivial challenges. The algorithm relies on fractional coordinate computation, multiplication of pixel values with fractional weights, and accumulation of intermediate results. In real hardware systems, these operations must be realized using fixed-point arithmetic, as floating-point units are often expensive in terms of area, power, and latency. Designers must therefore carefully select numerical representations, manage precision, and handle rounding and saturation effects, all while maintaining acceptable image quality.

Another key challenge is parameterization. Practical image scaling engines must support multiple input and output resolutions rather than being hard-coded for a single size. A parameterized design enables reuse across different products and applications but requires thoughtful architectural planning to avoid inefficient division operations, excessive logic usage or timing bottlenecks.

Participants are expected to implement a complete bilinear image scaling module in Verilog HDL that operates on a pre-stored image and produces a resized output image according to specified parameters.

**ELECTRONICS  
ENGINEERING  
SOCIETY**





**UDYAM' 26**

## **Input Image Specifications:**

- The input image dimensions are known at compile time.
- Supported image formats:
  - Grayscale: 1 channel
  - RGB: 3 channels
- Pixel representation:
  - 8-bit unsigned per channel
- The input image shall be stored as a hex or bin file, generated using a python script.
- Participants must interpret the image file correctly (depending on original dimensions of image) to reconstruct spatial image data.

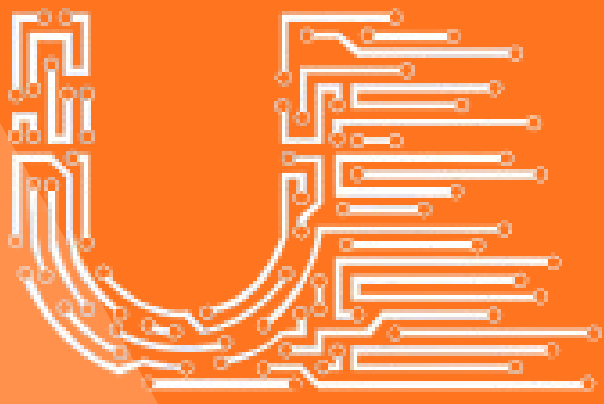
## **Output Image Specifications:**

- The output image resolution shall be parameterized in the design.
- The output image:
  - Must preserve the number of channels (grayscale or RGB)
  - Must use 8-bit unsigned pixel representation
- Output image must be written as a hex/bin file.

**ELECTRONICS  
ENGINEERING  
SOCIETY**







**UDYAM' 26**

## Scaling Method: Bilinear interpolation

For each output pixel, interpolation shall be performed using the four nearest neighboring pixels from the input image, following standard bilinear interpolation principles.

### Definitions:

- **(x<sub>out</sub>, y<sub>out</sub>)** - Coordinates of a pixel in the output image
- **(x<sub>in</sub>, y<sub>in</sub>)** - Corresponding real-valued coordinates in the input image
- **I(x, y)** - Grayscale intensity of the input image at integer pixel location (x, y)

### Input–Output Coordinate Mapping:

The input image resolution is fixed as:

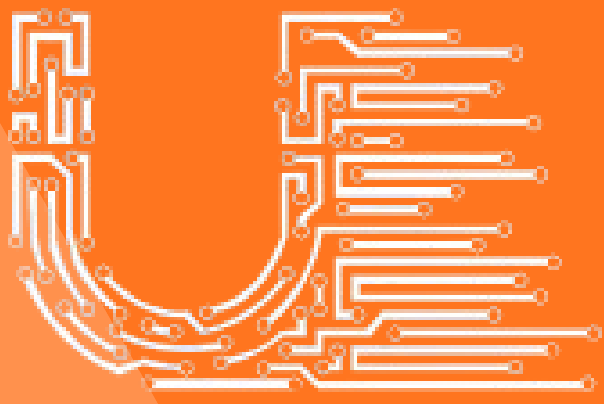
- Input Width = **W<sub>in</sub>**
- Input Height = **H<sub>in</sub>**

The output image resolution is parameterized as:

- Output Width = **W<sub>out</sub>**
- Output Height = **H<sub>out</sub>**

The mapping from output pixel coordinates to input pixel coordinates shall be computed as:

- **$x_{in} = x_{out} \times (W_{in} / W_{out})$**
- **$y_{in} = y_{out} \times (H_{in} / H_{out})$**



**UDYAM' 26**

where:

- $x_{out} \in [0, W_{out} - 1]$
- $y_{out} \in [0, H_{out} - 1]$

### **Neighbor Pixel Selection:**

- $x0 = \text{floor}(x_{in})$
- $y0 = \text{floor}(y_{in})$

The four neighboring input pixels used for interpolation are:

- $I00 = I(x0, y0)$
- $I10 = I(x0 + 1, y0)$
- $I01 = I(x0, y0 + 1)$
- $I11 = I(x0 + 1, y0 + 1)$

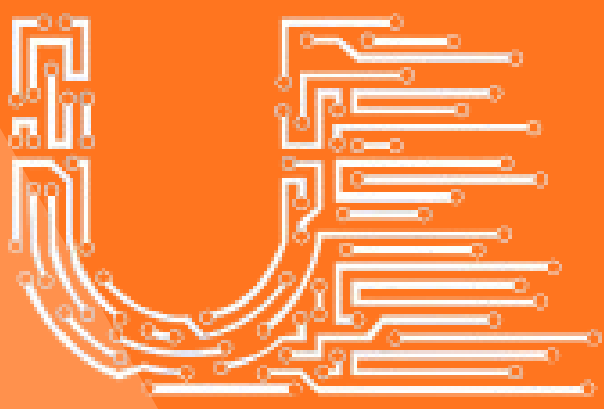
### **Define the fractional distances:**

- $a = x_{in} - x0$
- $b = y_{in} - y0$

where:

- $a \in [0, 1)$
- $b \in [0, 1)$





**UDYAM' 26**

## Bilinear Interpolation Equation:

The output pixel intensity  $I(x_{out}, y_{out})$  shall be computed as:

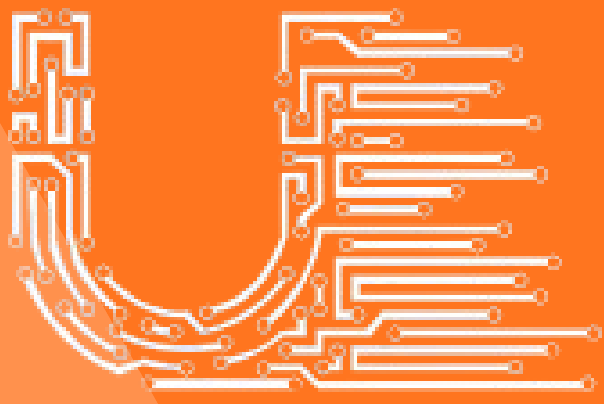
$$I(x_{out}, y_{out}) = (1 - a)(1 - b) * I_{00} + a(1 - b) * I_{10} + (1 - a)b * I_{01} + ab * I_{11}$$

## Implementation Constraints:

- Input and output pixels are 8-bit unsigned grayscale/RGB values.
- All arithmetic operations must use fixed-point or integer arithmetic only.
- Use of floating-point constructs is strictly prohibited.
- **Maximum internal fixed-point precision is 8 bits.**
- The design must work for both grayscale and RGB images.
- The architecture need not be streaming; full-frame storage is allowed.
- Internal implementation details (precision, rounding, scheduling) are left to the participant.
- Use of initial blocks and \$readmemh / \$writememh for memory initialization and output dumping is permitted inside the design module.
- Only synthesizable Verilog constructs (except file I/O) should be used.

**ELECTRONICS  
ENGINEERING  
SOCIETY**





**UDYAM' 26**

## **Design Expectations:**

Participants are encouraged to explore:

- Efficient computation of bilinear interpolation.
- Parallelism and pipelining where appropriate.
- Thoughtful register usage and data reuse.
- Novel architectural ideas that reduce latency, area, or complexity.
- Designs that scale well across different output resolutions.

**There is no single “correct” architecture – innovation is encouraged.**

## **Evaluation Criteria:**

Submissions will be evaluated based on the following criteria:

### **1. Image Quality:**

- PSNR (Peak Signal-to-Noise Ratio)
- SSIM (Structural Similarity Index)

### **2. Architectural Quality:**

- Novelty and originality of the processing architecture
- Efficiency of data handling and computation
- Logical organization and clarity of the design

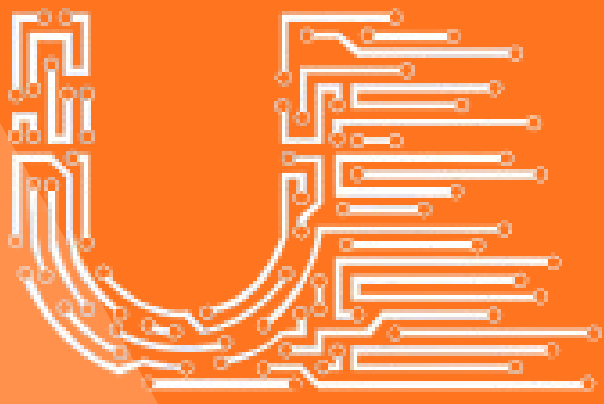
### **3. Resource Utilization:**

- Estimated hardware efficiency
- Avoidance of unnecessary computation or storage
- Clean and scalable design choices

**ELECTRONICS  
ENGINEERING  
SOCIETY**







**UDYAM' 26**

## **Prohibited Practices:**

- Hard-coding output image values.
- Using lookup tables containing precomputed output images.
- Using floating-point arithmetic or software models.
- Bypassing bilinear interpolation logic.
- Plagiarism of any sort.

**Violations may lead to penalties or disqualification.**

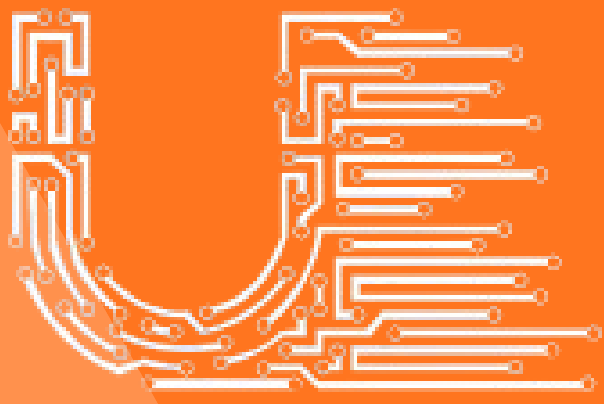
## **Deliverables:**

### **Participants must submit:**

1. Verilog HDL source code
2. A brief design description explaining:
  - Architectural approach
  - Handling of scaling and interpolation
  - Any optimization techniques used

**ELECTRONICS  
ENGINEERING  
SOCIETY**





**UDYAM' 26**

## **Prize Pool:**

**1<sup>st</sup> Place - ₹ 10,000**

**2<sup>nd</sup> Place - ₹ 7,500**

**3<sup>rd</sup> Place - ₹ 5,000**

**Separate prize pool for freshers:  
Goodies worth ₹ 5,000**

## **Contact us for any queries:**

Mayank Pratap Singh: **6395140156**

Vivek Premi: **9302195956**

**ELECTRONICS  
ENGINEERING  
SOCIETY**

